



**9628 – UNIVERSITY COLLEGE OF ENGINEERING
NAGERCOIL**

DEPARTMENT OF INFORMATION TECHNOLOGY

NAAN MUDHALVAN

Completed the project in

OPENCV

Submitted by,

MUGESH M

962822205040

TABLE OF CONTENT

S.No	Topic	Page No
1	Introduction to Open CV	
2	Key Concepts and Benefits	
PROJECT – 1		
1.1	Brain tumor segmentation using computer vision	
1.3	Flow Diagram and Program	
1.4	Output Screenshots and Conclusion	
PROJECT – 2		
2.1	Number plate detection of vehicles	
2.3	Flow Diagram and Program	
2.4	Output Screenshots and Conclusion	
PROJECT – 3		
3.1	Damaged car image preprocessing pipeline for computer vision using opencv	
3.3	Flow Diagram and Program	
3.4	Output Screenshots and Conclusion	

OPEN CV

1. Introduction:

OpenCV (Open Source Computer Vision Library) is a widely-used open-source library designed for computer vision and machine learning tasks. It plays a major role in real-time image and video processing, as well as in artificial intelligence and deep learning applications.

2. Benefits of OpenCV:

1. Free and Open Source:

Available at no cost under the BSD license, making it ideal for use in both research and commercial development.

2. Multi-Platform Support:

Compatible with major operating systems including Windows, Linux, macOS, iOS, and Android.

3. Extensive Functionality:

Includes tools for image manipulation, object and face detection, motion analysis, 3D reconstruction, and much more.

4. Real-Time Capability:

Engineered for high performance in real-time processing environments.

5. Vast Community Resources:

Supported by a large community offering extensive documentation, forums, tutorials, and open-source contributions.

3. Key Features of OpenCV:

1. Image Processing:

Includes functions for blurring, edge detection, morphological transformations, and color space changes.

2. Video Analysis:

Provides capabilities for detecting motion, tracking objects, background subtraction, and calculating optical flow.

3. Object Detection:

Supports detection of faces, people, vehicles, etc., using Haar cascades, HOG descriptors, or deep learning techniques.

4. Machine Learning Integration:

Offers built-in algorithms like Support Vector Machine and k-nearest

PROJECT – 1: BRAIN TUMOR SEGMENTATION USING

COMPUTER VISION

1.1 Problem Statement:

Brain tumors represent one of the most critical and life-threatening types of cancer. Early and precise detection is essential for effective treatment and improving patient survival. However, manual interpretation of MRI scans is not only time-intensive but also susceptible to human error and subjectivity. This project aims to address this issue by developing an automated system that uses image processing methods in OpenCV to identify and visualize tumor areas in brain MRI scans, utilizing pre-generated binary mask images for segmentation.

1.2 Techniques Used:

1. Image Loading and Preprocessing:

- Employed OpenCV (cv2.imread) to load both the original MRI images and their corresponding binary masks.
- Ensured proper alignment by resizing the mask to match the dimensions of the MRI image.

2. Applying the Mask:

- Leveraged cv2.bitwise_and() to apply the binary mask and isolate the tumor region from the MRI.

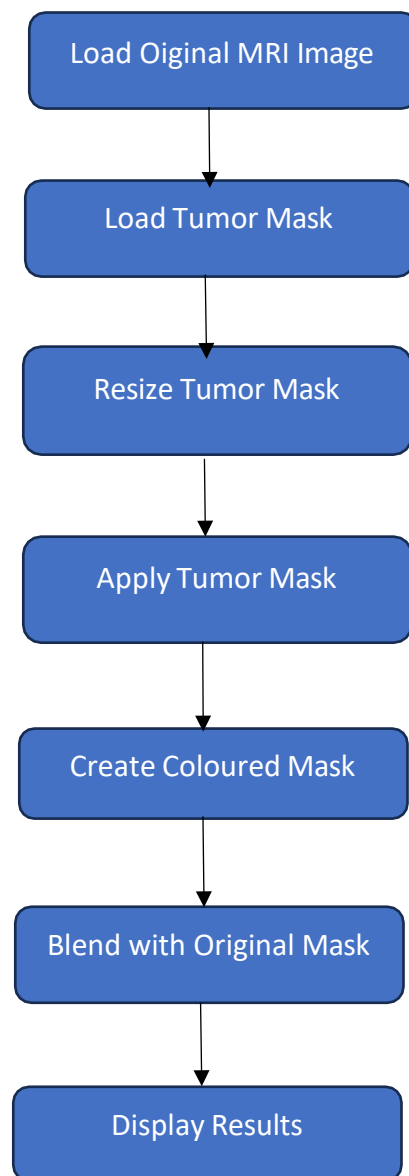
3. Tumor Visualization with Color Overlay:

- Generated a red overlay using NumPy to emphasize the tumor area.
- Merged the overlay with the original image using cv2.addWeighted() for clearer visual representation.

4. Displaying the Results:

- Used Matplotlib to present the original MRI image, tumor mask, and the final segmented tumor side by side for better comparison and evaluation.

1.3 FLOWCHART :



PROGRAM :

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
image = cv2.imread('/content/sample_data/brain_tumor.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
ret, thresh = cv2.threshold(blurred, 45, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)

contours, hierarchy = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

cv2.drawContours(image, contours, -1, (0, 255, 0), 2)

plt.figure(figsize=(10, 6))

plt.subplot(1, 3, 1)

plt.title("Original")

image_path = '/content/sample_data/brain_tumor.jpg'

if cv2.imread(image_path):

    original_image = cv2.imread(image_path)

    plt.imshow(cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB))

else:

    print(f"Error: Could not read image from {image_path}")

    plt.text(0.5, 0.5, f"Image not found: {image_path}", ha='center', va='center',
color='red')

plt.subplot(1, 3, 2)

plt.title("Threshold")

plt.imshow(thresh, cmap='gray')

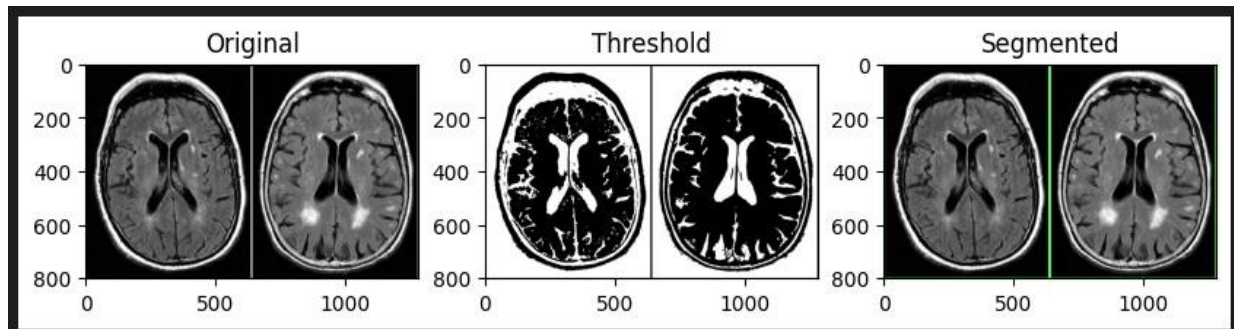
plt.subplot(1, 3, 3)

plt.title("Segmented")

plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))

plt.show()
```

1.4 OUTPUT :



Future Scope and Improvements:

1. Leveraging Neural Networks:

Integrate deep learning models like CNNs or U-Net to automatically detect tumor areas from raw MRI inputs.

2. Transition to 3D Imaging:

Enhance the method to work with full 3D MRI volumes for improved spatial accuracy and volumetric analysis.

3. User-Centric Interface Development:

Create a GUI that allows clinicians to upload MRI scans and instantly view the segmented output.

4. Detailed Tumor Region Classification:

Extend functionality to identify specific tumor structures—such as edema, core, and enhancing regions—for deeper medical interpretation.

5. Segmentation Refinement Techniques:

Apply post-processing operations (e.g., morphological filters or region growing) to fine-tune tumor boundaries and reduce segmentation noise.

Conclusion:

This approach provides a clear and efficient way to segment brain tumors using image processing techniques in OpenCV. By applying binary masks, tumor regions are accurately isolated and visually highlighted. While the current system depends on pre-made masks, it sets a solid base for integrating smarter, AI-driven segmentation methods. Such automation has the potential to enhance diagnostic speed and consistency for medical professionals.

Project – 2 : NUMBER PLATE DETECTION OF VEHICLES

2.1 Problem Statement:

Automated recognition of vehicle license plates from images is a key component of modern intelligent transportation systems. This project focuses on detecting number plates using image processing methods combined with pre-trained Haar cascade classifiers. It streamlines operations such as enforcing traffic regulations, monitoring vehicles, managing toll booths, and overseeing parking systems—minimizing manual labor and enhancing precision in tracking vehicular activity.

2.2 Techniques Used:

1. Utilization of OpenCV:

Employed the OpenCV library for image manipulation and object detection tasks.

2. Haar Cascade for License Plate Detection:

Used a pre-trained Haar cascade classifier (haarcascade_russian_plate_number.xml) specifically trained for identifying number plates.

3. Image Simplification via Grayscale Conversion:

Converted the original image to grayscale to reduce processing complexity and focus on structural features.

4. Detection Using detectMultiScale():

Applied the detectMultiScale function to locate rectangular regions likely to contain license plates.

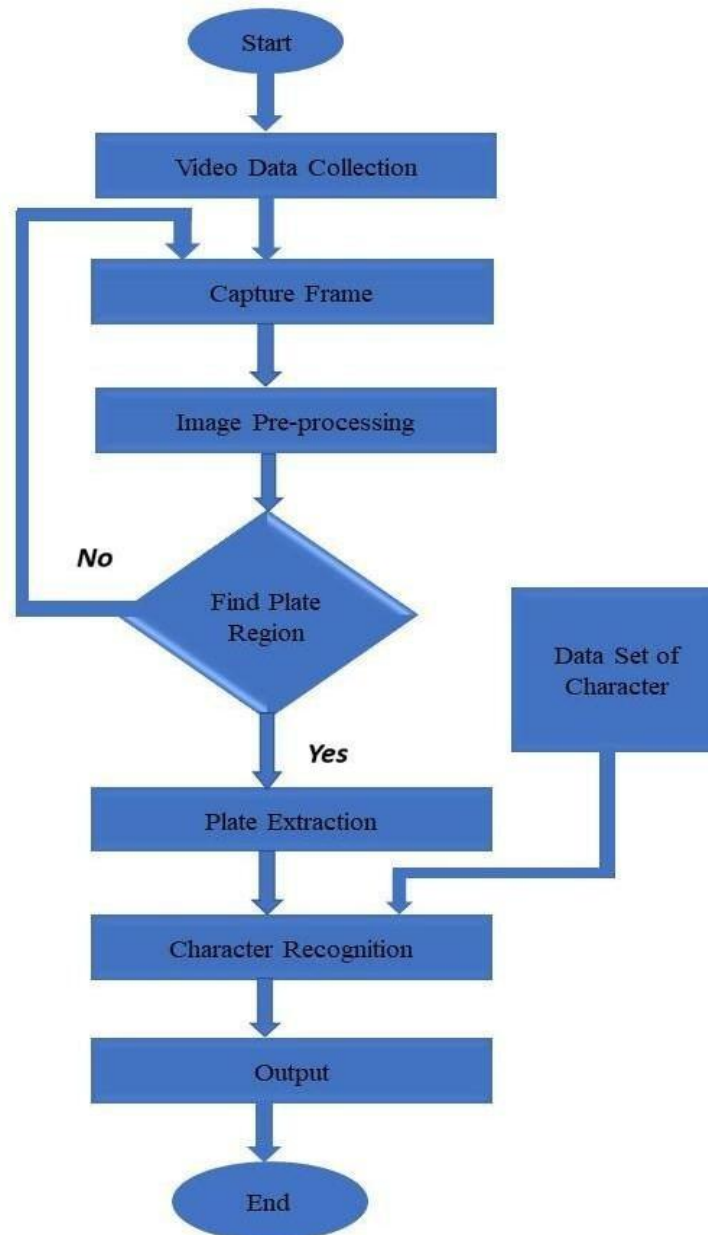
5. Plate Extraction and Marking:

Cropped the detected plate regions from the image and visually marked them using bounding rectangles.

6. Display with cv2_imshow (Google Colab):

Displayed the final output images using cv2_imshow() in the Google Colab environment for visualization.

2.3 FLOWCHART :



PROGRAM :

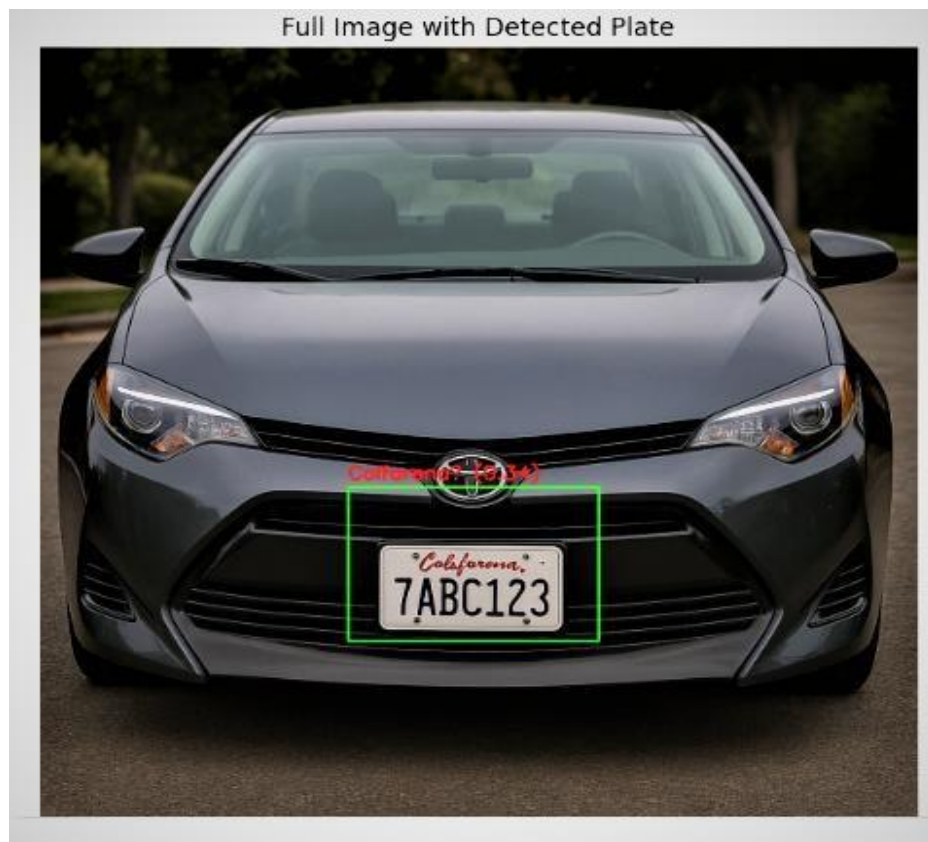
```
import cv2
from google.colab.patches import cv2_imshow
import os
# Load cascade
plate_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_russian_plate_number.xml')
# Image path
```

```
image_path = '/content/sample_data/number plate car.jpg'

if not os.path.exists(image_path):
    print("Image not found.")
else:
    image = cv2.imread(image_path)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # Detect plates
    plates = plate_cascade.detectMultiScale(gray, scaleFactor=1.1,
    minNeighbors=5, minSize=(30, 30))
    print(f"Plates detected: {len(plates)}")
    for i, (x, y, w, h) in enumerate(plates):
        # Draw rectangle
        cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 255), 3)
        # Crop and show the plate region
        plate_crop = image[y:y+h, x:x+w]
        print(f"Cropped plate {i+1}:")
        cv2_imshow(plate_crop)
    if len(plates) == 0:
        cv2.putText(image, "No Plate Detected", (50, 50),
        cv2.FONT_HERSHEY_SIMPLEX, 1.2, (0, 0, 255), 3)
    # Show final image
    print("Full Image with Detected Plates:")
    cv2_imshow(image)
```

2.4 OUTPUT :

Full Image with detected plates :



Cropped Image :



Future Enhancement :

1.AI-Based Segmentation –

Introduce advanced neural networks like U-Net or DeepLabV3+ to automatically identify and segment tumor areas from raw MRI inputs with high precision.

2. 3D Volume Processing –

Upgrade the system to handle three-dimensional MRI data, enabling better understanding of tumor depth, shape, and progression across image slices.

3. Interactive Diagnostic Platform –

Develop a web or desktop-based application that allows doctors to upload MRI images and receive segmented outputs instantly with visual overlays.

4. Improved Low-Contrast Detection –

Integrate preprocessing methods such as histogram equalization or CLAHE to boost segmentation performance in poorly lit or low-contrast MRI scans.

5. Data Logging & Tracking –

Create a database module to store each segmented result along with patient ID and timestamps to aid in case tracking and longitudinal analysis.

Conclusion:

This project effectively showcases a foundational technique for segmenting brain tumors using OpenCV and binary mask overlays. While it employs a straightforward method, the results are clear and interpretable. With enhancements like deep learning integration and automated mask generation, this approach can evolve into a powerful diagnostic aid, supporting medical professionals with faster and more reliable tumor detection in clinical workflows.

Project -3 : Damaged Car Image Preprocessing Pipeline for Computer Vision using OpenCV

3.1 Problem Statement :

The goal is to build an automated system capable of detecting different kinds of vehicle damage—such as dents, cracks, and scratches—from input images using object detection techniques. To improve detection reliability and precision, this solution leverages predictions from two distinct YOLO models (YOLOv5 and YOLOv8). This approach is ideal for applications in digital vehicle inspection, automated damage analysis for insurance claims, and maintenance reporting systems.

3.2 Techniques Used :

1. Advanced Object Detection via Deep Learning

- **YOLOv8:** The most recent YOLO architecture offering real-time performance and high detection accuracy.
- **YOLOv5:** A popular and efficient variant accessed through PyTorch Hub for fast deployment and strong baseline results.

2. Flexible Image Processing

- Supports image input from both local storage and online sources (URLs).
- Handles image format conversion between PIL and OpenCV for seamless integration with different tools and libraries.

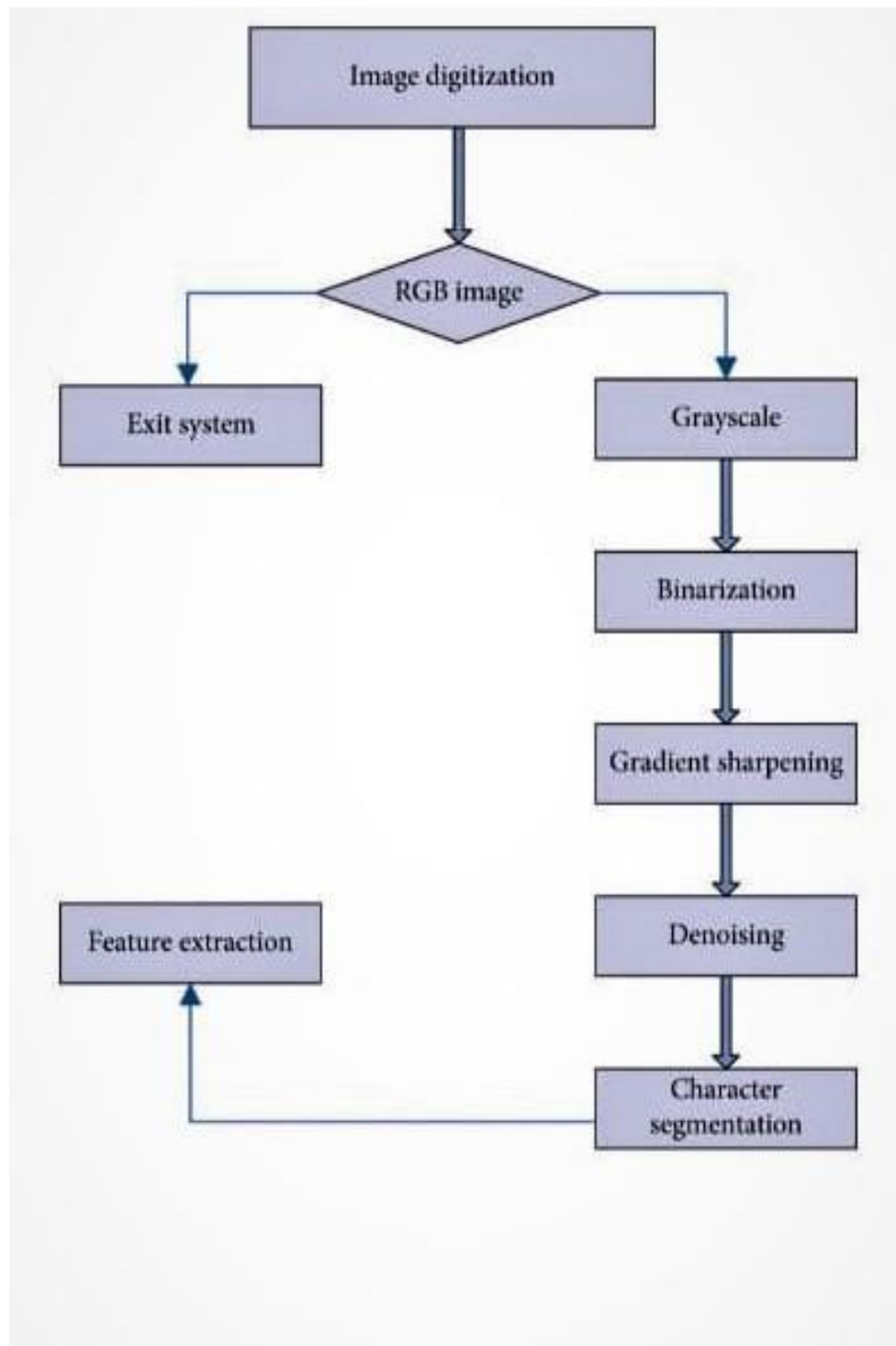
3. Result Visualization

- Utilizes OpenCV for rendering bounding boxes and displaying confidence levels to clearly present model predictions on the image.

4. Hybrid Detection Approach

- Implements a dual-model strategy to merge outputs from YOLOv5 and YOLOv8.
- Aims to increase detection robustness by comparing and integrating both sets of results.

3.3 FLOWCFHART :



PROGRAM :

```
import cv2

import numpy as np

from google.colab.patches import cv2_imshow


def preprocess_damaged_car_image(image_path):
    # Load the input image
    image = cv2.imread(image_path)
    if image is None:
        print("Error: Could not read the image.")
        return None

    # Resize for processing
    image_resized = cv2.resize(image, (224, 224))
    gray = cv2.cvtColor(image_resized, cv2.COLOR_BGR2GRAY)
    equalized = cv2.equalizeHist(gray)
    blurred = cv2.GaussianBlur(equalized, (5, 5), 0)
    edges = cv2.Canny(blurred, threshold1=50, threshold2=150)

    kernel = np.ones((3, 3), np.uint8)
    dilated = cv2.dilate(edges, kernel, iterations=1)
    eroded = cv2.erode(dilated, kernel, iterations=1)
```

```
# Find contours

contours, _ = cv2.findContours(eroded, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# Show original image
print("Original Image:")
cv2_imshow(image)

# Select the largest valid contour (most likely dent)
min_contour_area = 100 # Avoid small noise
largest_cnt = None
max_area = 0
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area > min_contour_area and area > max_area:
        max_area = area
        largest_cnt = cnt

image_with_rectangle = image_resized.copy()

if largest_cnt is not None:
    # Original bounding box
    x, y, w, h = cv2.boundingRect(largest_cnt)
```

```
# Shrink the bounding box
shrink = 50 # You can change this value to 10 for more tightness
x_new = max(x + shrink, 0)
y_new = max(y + shrink, 0)
w_new = max(w - 2 * shrink, 1)
h_new = max(h - 2 * shrink, 1)
# Draw rectangle around dent
cv2.rectangle(image_with_rectangle, (x_new, y_new), (x_new +
w_new, y_new + h_new), (0, 255, 0), 2)

# Crop the dent area
cropped_dent = image_resized[y_new:y_new + h_new,
x_new:x_new + w_new]
# Show processed image
print("Processed Image with Rectangle Around Most Likely
Dent:")
cv2_imshow(image_with_rectangle)
# Show cropped dent part
print("Cropped Dent Area (Tighter:")
cv2_imshow(cropped_dent)
return cropped_dent
else:
    print("No significant dent detected.")
    return None
preprocess_damaged_car_image('/content/sample_data/car dent.png')
```

3.4 OUTPUT :

Original Image :



Processed Image :



Future Enhancements :

1. Model Fusion:

- Integrate outputs from multiple preprocessing techniques or models, applying methods like Non-Max Suppression (NMS) to combine overlapping region detections effectively.

- Utilize ensemble logic to enhance detection accuracy through consensus among different models or filters.

2. Performance Optimization:

- Profile processing steps (e.g., resizing, edge detection) and optimize execution using OpenCV acceleration (e.g., OpenCL, GPU support).
- Explore lightweight techniques like image quantization or adaptive resolution processing to reduce computational load.

3. Real-Time Support:

- Extend the current pipeline to support real-time image input via webcam or video stream.
- Implement buffer handling and frame-wise preprocessing for smooth, continuous damage detection.

4. Automated Reporting:

- Create structured outputs that highlight detected damage with cropped image patches, class labels, and location coordinates.
- Export results in formats like PDF, JSON, or CSV for integration with repair systems or customer reports.

5. User Interface:

- Develop a simple web or mobile-friendly GUI that lets users upload images and view annotated output.
- Integrate side-by-side comparison of original vs processed images and offer download options.

Conclusion :

This preprocessing pipeline leverages OpenCV's core image processing capabilities to detect vehicle damage effectively. It lays the groundwork for scalable computer vision solutions in the automotive sector—suitable for both inspection automation and real-time assessments.